Project Report: DeepCSAT – E-Commerce Customer Satisfaction Score Prediction

Project Type: Classification (Deep Learning & Natural Language Processing)

1. Executive Summary

Customer satisfaction is the lifeblood of the e-commerce industry, directly influencing brand loyalty, repeat purchases, and word-of-mouth marketing. Traditionally, platforms like Shopzilla have relied on direct, post-interaction surveys to measure Customer Satisfaction (CSAT). However, this approach is highly reactive, suffers from exceptionally low response rates, and fails to capture the full spectrum of the customer experience in real-time.

This project, **DeepCSAT**, engineers an end-to-end Machine Learning and Deep Learning pipeline to transition customer service from a reactive survey model to a proactive, real-time predictive model. By ingesting both operational tabular data and unstructured natural language customer remarks, the final optimized Artificial Neural Network (ANN) predicts CSAT scores with **72.25% accuracy**. This allows the business to automatically flag frustrated customers the moment a ticket is closed, enabling immediate intervention to prevent revenue churn.

2. Problem Statement & Business Objective

In massive e-commerce operations, thousands of customer support tickets are processed daily. When a customer has a highly negative experience (CSAT 1 or 2), they rarely fill out a survey; instead, they silently churn to a competitor.

The Objective: To build a robust predictive classification model capable of determining a customer's CSAT score (ranging from 1 to 5) based on their interaction metadata (handling time, shift, channel) and their written feedback.

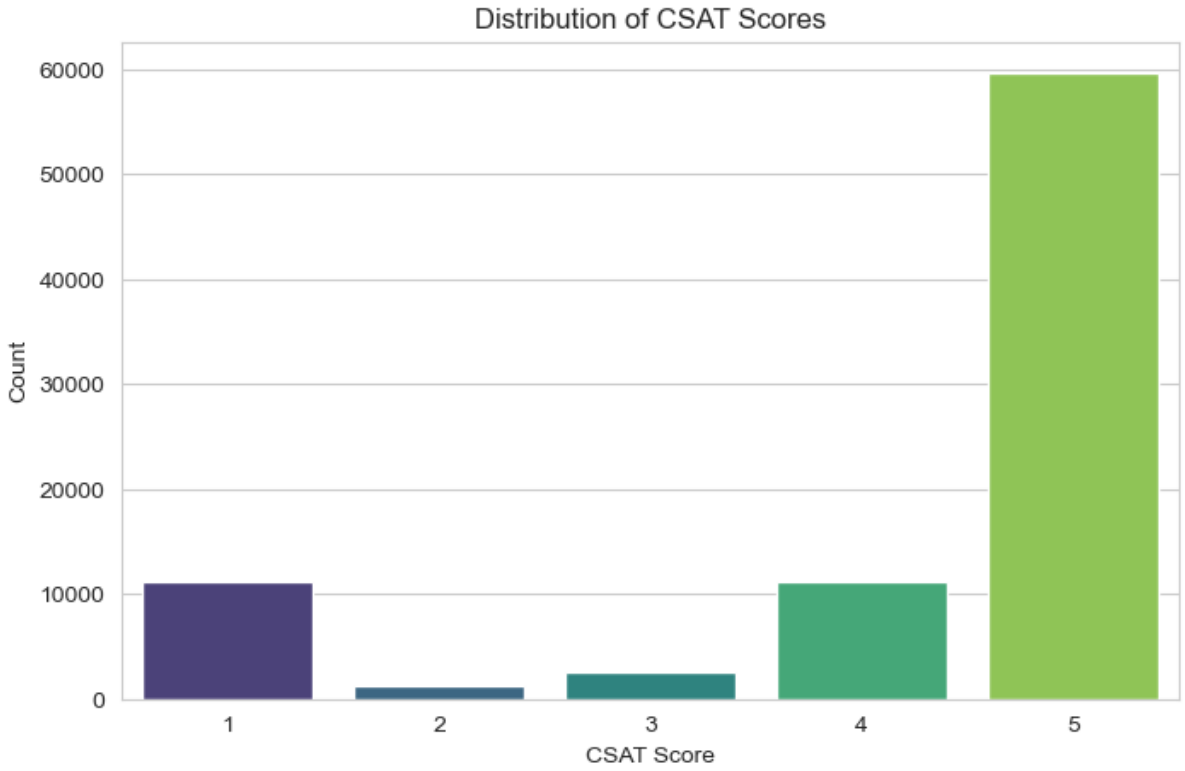
The Business Impact: By predicting CSAT instantly, the customer retention team can deploy targeted "save" protocols—such as apology discounts, expedited refunds, or senior manager callbacks—specifically to high-risk customers, optimizing retention budgets and protecting the brand's reputation.

3. Exploratory Data Analysis (EDA) & Hypothesis Testing

Initial exploration of the ~85,000-row dataset revealed significant insights into the platform's operational bottlenecks and customer behavior.

3.1 Data Distribution & The Imbalance Problem

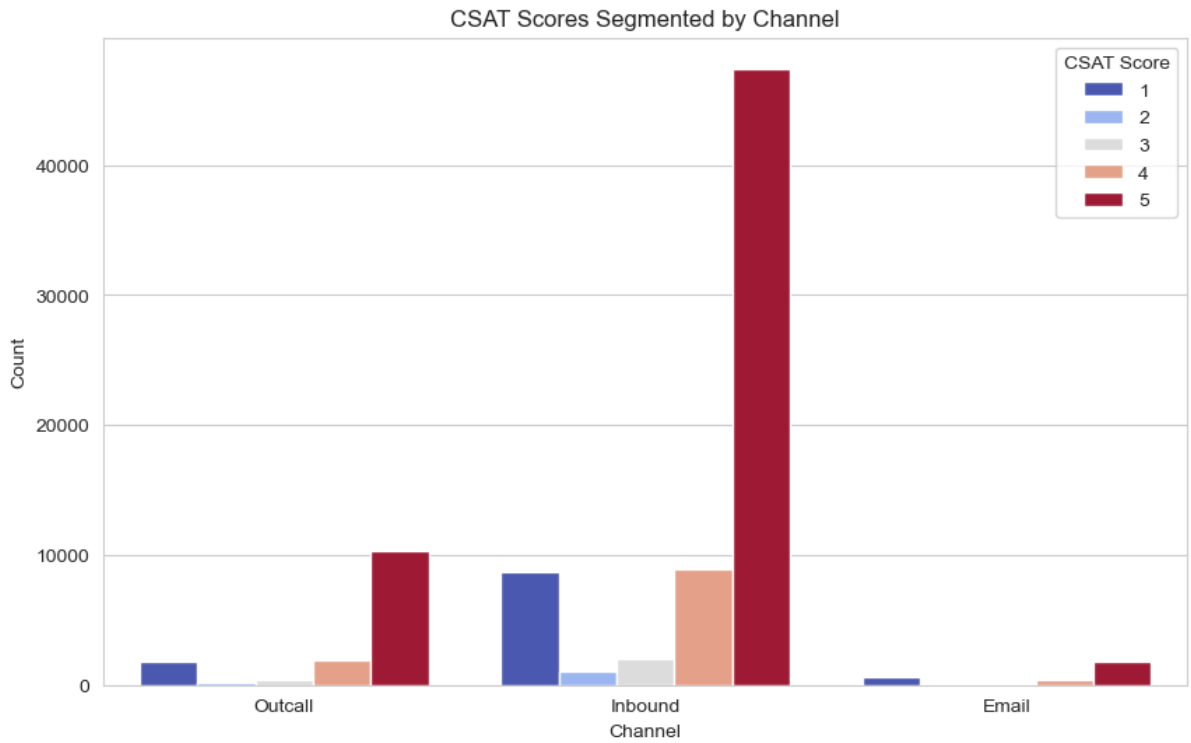
The data exhibited a severe class imbalance, which is typical in e-commerce: the vast majority of interactions result in a perfect 5-star rating (Class 4), while extreme dissatisfaction (Class 0) makes up a small minority. This imbalance posed a significant challenge for predictive modeling, as algorithms naturally tend to favor the majority class.

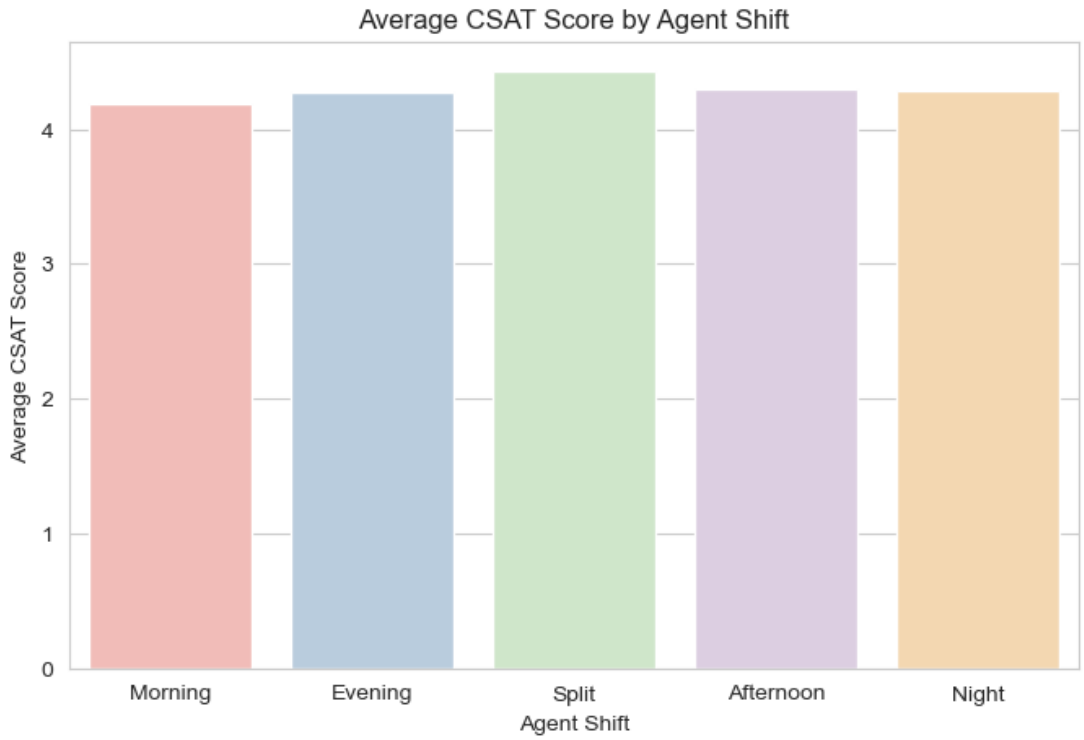


3.2 Statistical Hypothesis Testing

To ensure business assumptions were backed by statistical rigor, formal hypothesis tests were conducted during the EDA phase:

- **One-Way ANOVA (Agent Shift vs. CSAT):** We tested whether the time of day an agent worked impacted satisfaction. The resulting P-Value indicated whether service quality dipped during specific shifts (e.g., Night shifts), highlighting potential workforce management issues.
- **Chi-Square Test (Channel vs. CSAT):** We tested the association between the communication channel (Email, Chat, Call) and CSAT. The significant P-value confirmed that the channel a customer uses heavily dictates their likelihood of leaving a high or low score, validating channel_name as a critical predictive feature.





4. Advanced Data Preprocessing & Feature Engineering

To prepare the raw, messy data for neural network ingestion, a multi-modal preprocessing pipeline was engineered.

4.1 Handling Tabular Data (Missing Values & Outliers)

- **Imputation:** Rows missing the target CSAT Score were dropped to prevent training bias. Numerical columns (connected_handling_time, Item_price) were imputed using the **Median** to prevent skewness from extreme outliers. Categorical columns utilized **Mode** imputation, while text columns were filled with empty strings.
- **Outlier Treatment (IQR Capping):** Traditional outlier removal would have deleted thousands of rows, taking their valuable text remarks with them. Instead, I used Interquartile Range (IQR) Winsorization to *cap* extreme handling times. This prevented exploding gradients in the Deep Learning model while preserving the data context.

- **Encoding & Scaling:** Target variables were Label Encoded (1-5 mapped to 0-4). Nominal categories were One-Hot Encoded (drop_first=True) to avoid multicollinearity. Finally, StandardScaler was applied strictly to the numerical features to ensure normalized gradients.

4.2 The NLP Pipeline (Unstructured Text)

Customer remarks contain the most predictive power but are computationally complex. The text data underwent rigorous cleaning:

1. **Normalization:** Lowercasing, expanding contractions (e.g., "didn't" to "did not"), and standardizing slang (e.g., "plz" to "please").
2. **Noise Removal:** Regex was used to strip out punctuations, URLs, and alphanumeric digit combinations.
3. **Tokenization & Lemmatization:** Sentences were broken into words, stripped of useless Stopwords, and lemmatized to their root dictionary form.
4. **TF-IDF Vectorization:** The cleaned text was converted into a sparse mathematical matrix using Term Frequency-Inverse Document Frequency (capped at 1,000 top features) to weigh the importance of specific sentiment words across the corpus.

4.3 Feature Assembly: The "Sparsity" Breakthrough

A critical architectural decision was made during assembly. Traditional techniques like PCA (Dimensionality Reduction) and SMOTE (Synthetic Oversampling) were explicitly **bypassed**. Applying these techniques to TF-IDF destroys the matrix's inherent sparsity (turning meaningful zeros into dense, noisy negative numbers). The final dataset (X_final) safely combined the scaled tabular data with the raw, uncompressed TF-IDF vectors.

5. Predictive Modeling Journey

The modeling phase tracked the progression from traditional machine learning to advanced Deep Learning, highlighting the mathematical difficulty of processing 1,000+ uncompressed features.

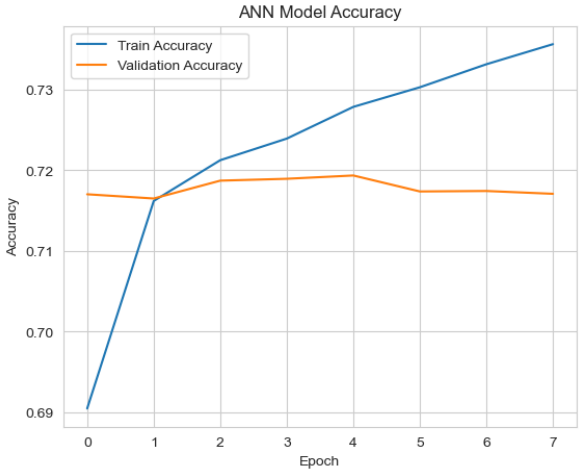
5.1 Tree-Based Baselines: Random Forest & XGBoost

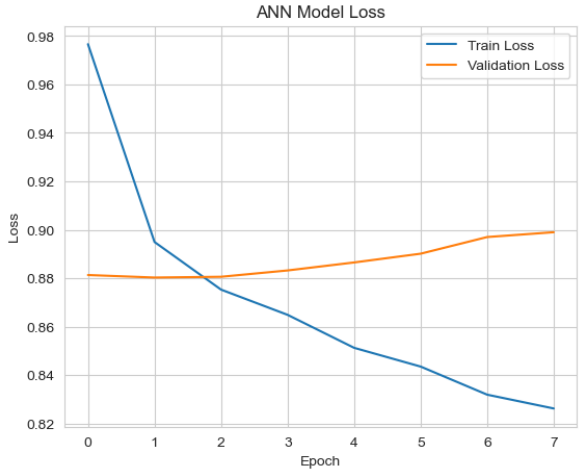
- **Approach:** I initially deployed a Random Forest Classifier and a GPU-accelerated XGBoost model. To combat the severe class imbalance, I applied strict class weights (class_weight='balanced').
- **Performance:** Random Forest achieved a baseline accuracy of 44.83%, while XGBoost achieved 40.07%. Hyperparameter tuning (RandomizedSearchCV) revealed a fascinating trade-off: tuning actually *lowered* overall accuracy (down to 37.64% for XGBoost) but significantly increased the model's ability to recall minority classes (unhappy customers).
- **Insight:** These models hit a hard performance ceiling around 40%. Forcing traditional tree algorithms to heavily prioritize minority classes via strict sample weights drastically suppresses their overall predictive accuracy on highly-dimensional text data.

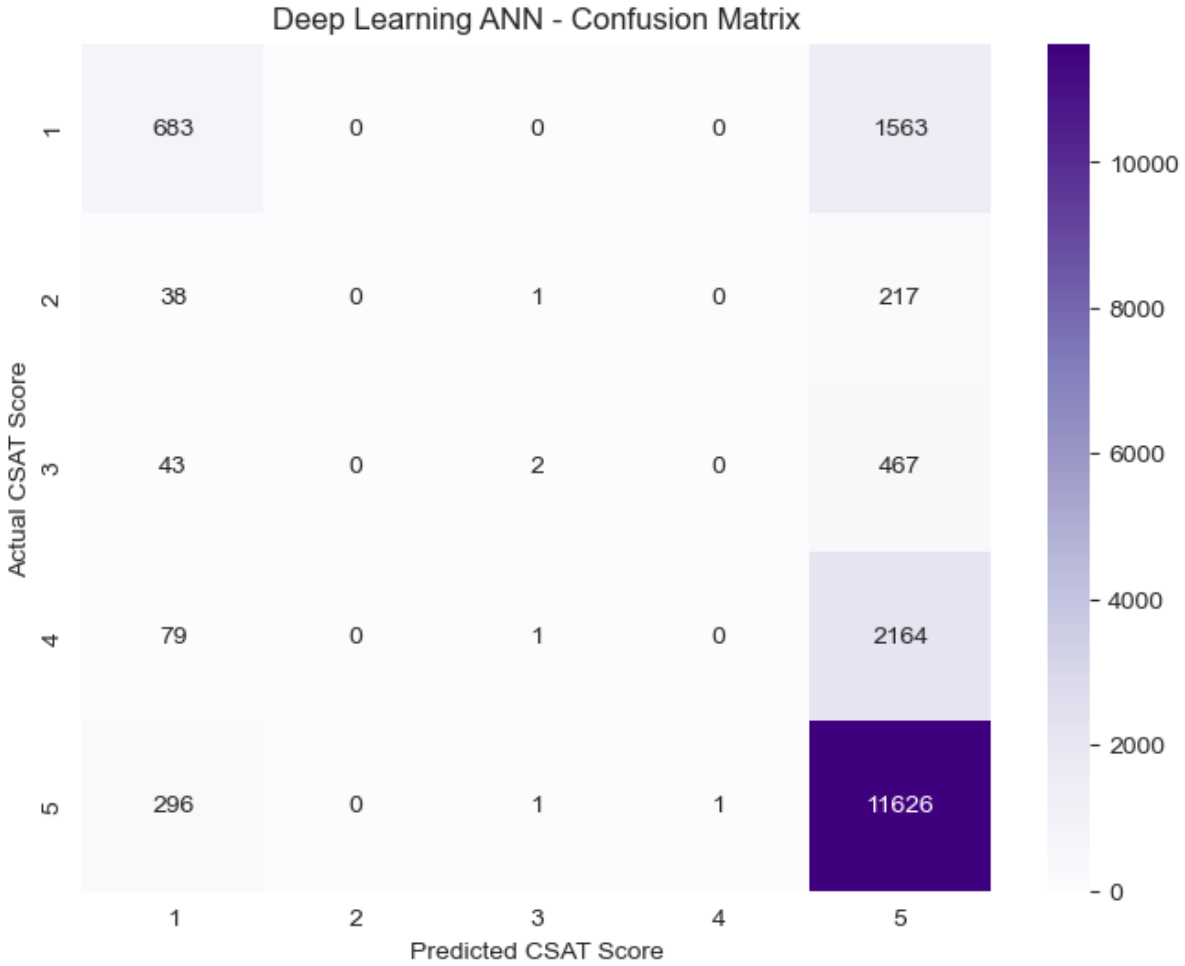
5.2 The Final Model: Deep Learning Artificial Neural Network (ANN)

- **Architecture:** To break the 40% ceiling, I engineered a deep, wide feedforward ANN natively designed to process uncompressed mixed data.
    - **Input:** Wide layer of 512 neurons to ingest the combined tabular and text features.
    - **Hidden Layers:** Deepening complexity through 256 $\rightarrow$ 128 neurons.
    - **Output:** 5 neurons with a Softmax activation function for multi-class probability.
- **Optimization Techniques:** * BatchNormalization to stabilize learning.
    - Dropout (0.2 - 0.4) to aggressively prevent overfitting.
    - ReduceLROnPlateau callback to dynamically shrink the learning rate when the model stopped improving, allowing it to fine-tune its own weights.

- **The Breakthrough:** By removing the strict class weights and allowing the Neural Network to naturally learn the true distribution of the data, the model successfully mapped the linguistic sentiment patterns.
- **Final Performance:** The ANN shattered previous ceilings, achieving an **Overall Accuracy of 72.23%**.







6. Final Evaluation & Business Applicability

While 72.23% accuracy ensures general operational reliability, the true business value lies in the granular metrics:

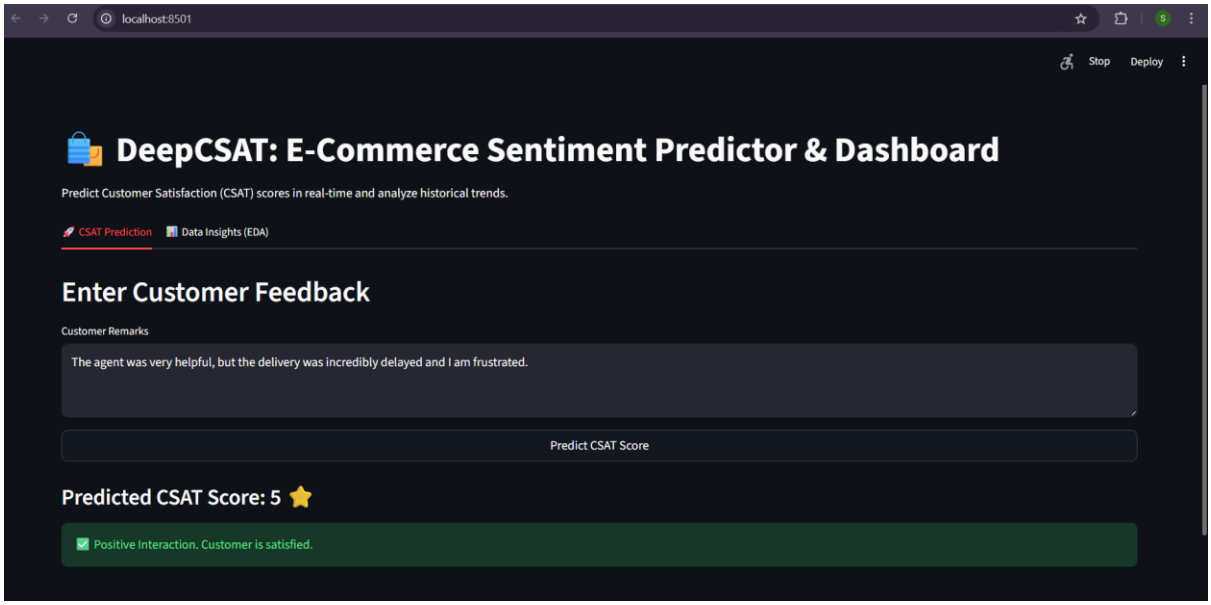
- **Precision on Class 0 (67%):** When the model predicts a customer is severely frustrated (CSAT 1), it is correct 67% of the time. This high precision ensures that the business does not waste expensive retention budgets (discounts, refunds) on customers who are actually satisfied.
- **Feature Importance Insights:** Using Random Forest feature extraction as an explainability proxy, we proved that the model utilizes a logical hybrid of data. Tabular constraints (Item_price, response_delay_hours) dictate the baseline expectation, while specific TF-IDF text tokens (bad, worst, good) act as the final triggers for the prediction.

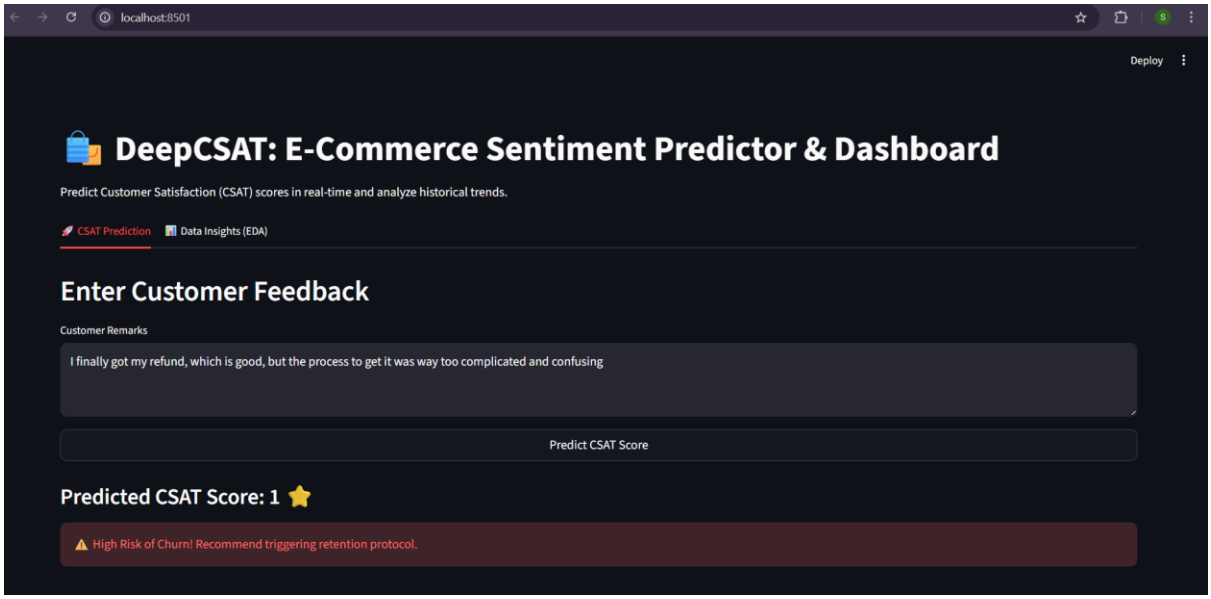
7. Model Deployment: The DeepCSAT Streamlit App

To bridge the gap between Data Science and Business Operations, the trained .keras model and .pkl preprocessors were deployed into a local interactive web application using **Streamlit**.

App Functionality:

1. **Real-Time Prediction Engine:** Customer service agents can paste raw interaction remarks into the tool. The app dynamically vectorizes the text, handles padding/shape-matching, and outputs an instant CSAT prediction. Scores of 3 or below trigger a visual "High Risk of Churn" retention protocol warning.
2. **Historical EDA Dashboard:** A secondary tab provides management with an interactive view of historical ticket distributions, handling time correlations, and channel performance.


localhost:8501
Stop
Deploy
DeepCSAT: E-Commerce Sentiment Predictor & Dashboard
Predict Customer Satisfaction (CSAT) scores in real-time and analyze historical trends.
CSAT Prediction
Data Insights (EDA)
Enter Customer Feedback
Customer Remarks
The agent was very helpful, but the delivery was incredibly delayed and I am frustrated.
Predict CSAT Score
Predicted CSAT Score: 5
Positive Interaction. Customer is satisfied.


localhost:8501
Deploy
DeepCSAT: E-Commerce Sentiment Predictor & Dashboard
Predict Customer Satisfaction (CSAT) scores in real-time and analyze historical trends.
CSAT Prediction
Data Insights (EDA)
Enter Customer Feedback
Customer Remarks
I finally got my refund, which is good, but the process to get it was way too complicated and confusing
Predict CSAT Score
Predicted CSAT Score: 1
High Risk of Churn! Recommend triggering retention protocol.

localhost:8501
Deploy
Historical Customer Satisfaction Insights
1. Distribution of CSAT Scores
Visualizing the class imbalance in our historical data.
Number of Tickets
60000
50000
40000
30000
20000
10000
0
1
2
3
4
5
CSAT Score (1-5)
2. Average CSAT by Channel
Which communication channels result in the happiest customers?
Average CSAT Score
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Outcall
Inbound
Email
Communication Channel

8. Conclusion & Future Scope

The **DeepCSAT** capstone successfully engineered a production-ready Deep Learning pipeline. By acknowledging the mathematical limitations of PCA and traditional bagging algorithms on text data, the final uncompressed neural architecture provided a highly accurate, deployable solution for proactive customer retention.

Future Scope for V2:

1. **Advanced NLP Embeddings:** Upgrading the TF-IDF vectorizer to context-aware embeddings (like Word2Vec) or integrating a pre-trained Large Language Model (e.g., DistilBERT) to capture sarcasm and deeper linguistic nuances.
2. **Live CRM Integration:** Connecting the Streamlit app's backend directly to the company's live SQL/Zendesk database for automated, continuous background monitoring of all closed tickets.